

6. Decision Trees

Machine learning

Jiri Chmelik

Department of Biomedical Engineering

Introduction

- Bias-Variance Tradeoff and Noise
- Decision Trees
 - basic principle
 - terminology
- CART (Classification And Regression Trees) Breiman et al. (1984)
 - splitting criterion (impurity measure)
 - stop-splitting rule
 - leaf assignment rule
- Bootstrap aggregated (bagged) trees
- Random Forest
- Adaptive Boosting (AdaBoost)
- Feature Importance with Decision Trees and Random Forest
- Another CART modifications – ID3 (Iterative Dichotomiser 3), C4.5, C5.0, CHAID (Chi-squared Automatic Interaction Detection), MARS (Multivariate Adaptive Regression Splines), etc.

Introduction

- Bias-Variance Tradeoff and Noise
- Decision Trees
 - basic principle
 - terminology
- CART (Classification And Regression Trees) Breiman et al. (1984)
 - splitting criterion (impurity measure)
 - stop-splitting rule
 - leaf assignment rule
- Bootstrap aggregated (bagged) trees
- Random Forest
- Adaptive Boosting (AdaBoost)
- Feature Importance with Decision Trees and Random Forest
- Another CART modifications – ID3 (Iterative Dichotomiser 3), C4.5, C5.0, CHAID (Chi-squared Automatic Interaction Detection), MARS (Multivariate Adaptive Regression Splines), etc.

Bias-Variance Tradeoff and Expected Test Error

$T = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$ n samples of training data T

$\mathbf{x}_i \in \mathbb{R}$ $y_i \in \mathbb{R}$ *i.i.d.* $P(X, Y)$

Expected Label (given feature vector $\mathbf{x}$):

$$\bar{y}(\mathbf{x}) = E_{y|\mathbf{x}}[Y] = \int_y y P(y|\mathbf{x}) dy$$

Training process:

$$h_T = \mathcal{A}(T)$$

Expected Test Error (given trained model h_T):

$$E_{(\mathbf{x}, y) \sim P}[(h_T(\mathbf{x}) - y)^2] = \iint_{xy} (h_T(\mathbf{x}) - y)^2 P(\mathbf{x}, y) dy d\mathbf{x}$$

Expected Classifier (given machine learning algorithm $\mathcal{A}$):

$$\bar{h} = E_{T \sim p^n}[h_T] = \int_T h_T P(T) dT$$

Bias-Variance Tradeoff and Expected Test Error

Expected Test Error (given $\mathcal{A}$):

$$E_{\substack{(\mathbf{x}, y) \sim P \\ T \sim P^n}}[(h_T(\mathbf{x}) - y)^2] = \iiint_{T\mathbf{x}y} (h_T(\mathbf{x}) - y)^2 P(\mathbf{x}, y) P(T) dy d\mathbf{x} dT$$

$$\begin{aligned} E_{x,y,T}[(h_T(\mathbf{x}) - y)^2] &= E_{x,y,T} \left[\left[(h_T(\mathbf{x}) - \bar{h}(\mathbf{x})) + (\bar{h}(\mathbf{x}) - y) \right]^2 \right] \quad (a+b)^2 = a^2 + 2ab + b^2 \\ &= E_{x,T} \left[(h_T(\mathbf{x}) - \bar{h}(\mathbf{x}))^2 \right] + 2 E_{x,y,T} \left[(h_T(\mathbf{x}) - \bar{h}(\mathbf{x})) (\bar{h}(\mathbf{x}) - y) \right] + E_{x,y} \left[(\bar{h}(\mathbf{x}) - y)^2 \right] \end{aligned}$$

$$E_{x,y,T}[(h_T(\mathbf{x}) - y)^2] = E_{x,T} \left[\left(h_T(\mathbf{x}) - \bar{h}(\mathbf{x}) \right)^2 \right] + E_{x,y} \left[\left(\bar{h}(\mathbf{x}) - y \right)^2 \right]$$

Bias-Variance Tradeoff and Expected Test Error

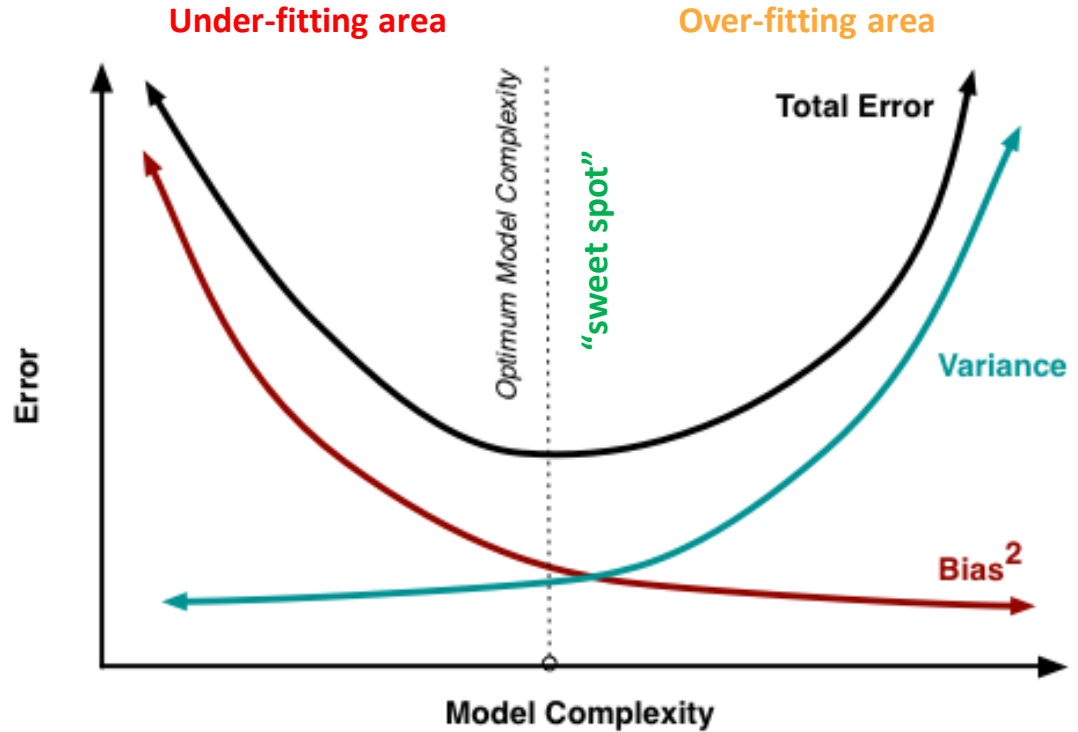
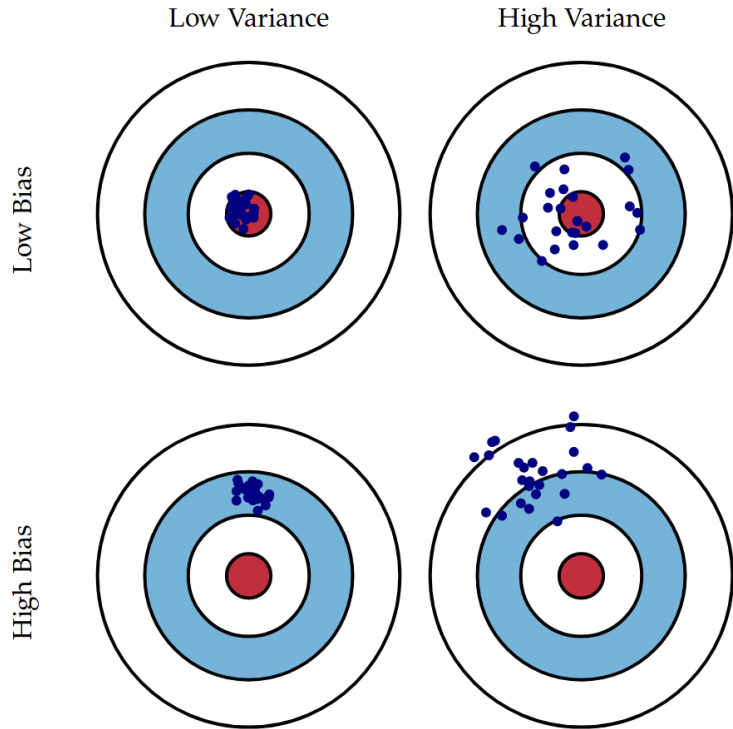
$$\begin{aligned}
 E_{x,y} [(\bar{h}(x) - y)^2] &= E_{x,y} \left[\left[(\bar{h}(x) - \bar{y}(x)) + (\bar{y}(x) - y) \right]^2 \right] && \text{= 0} \\
 &= E_x \left[(\bar{h}(x) - \bar{y}(x))^2 \right] + 2E_{x,y} [(\bar{h}(x) - \bar{y}(x))(\bar{y}(x) - y)] + E_{x,y} [(\bar{y}(x) - y)^2] && (a+b)^2 = a^2 + 2ab + b^2
 \end{aligned}$$

= 0

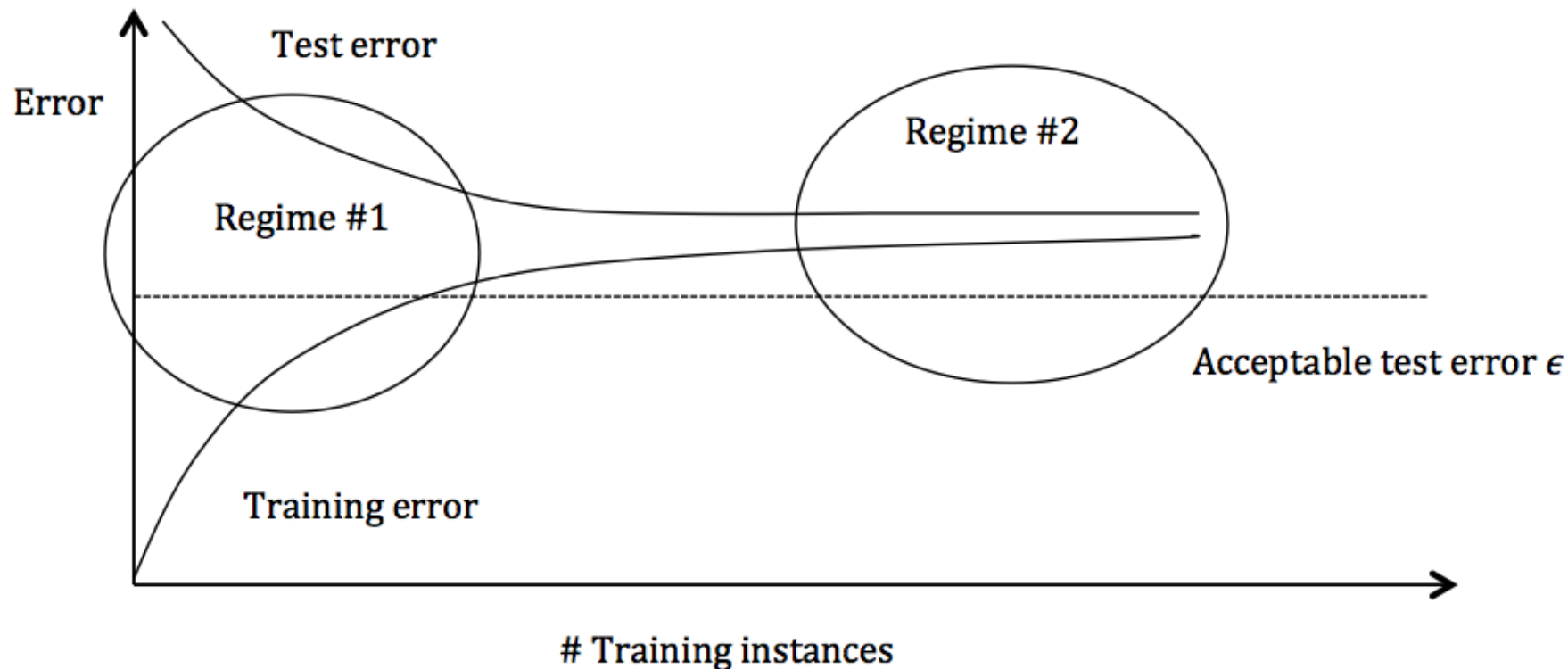
$$E_{x,y,T} [(h_T(x) - y)^2] = E_{x,T} [(h_T(x) - \bar{h}(x))^2] + E_{x,y} [(\bar{h}(x) - y)^2]$$

$$\underbrace{E_{x,y,T} [(h_T(x) - y)^2]}_{\text{Expected Test Error}} = \underbrace{E_{x,T} [(h_T(x) - \bar{h}(x))^2]}_{\text{Variance}} + \underbrace{E_x [(\bar{h}(x) - \bar{y}(x))^2]}_{\text{Bias}^2} + \underbrace{E_{x,y} [(\bar{y}(x) - y)^2]}_{\text{Noise}}$$

Bias-Variance Tradeoff and Expected Test Error



Bias-Variance Tradeoff and Expected Test Error



Bias-Variance Tradeoff and Expected Test Error

REGIME 1 (High Variance)

How to recognize?

- Training error is much lower than test error,
- Training error is lower than ϵ ,
- Test error is higher than ϵ .

Suggestions:

- Add more training data,
- Reduce model complexity,
- Bootstrap Aggregation („bagging“).

Bias-Variance Tradeoff and Expected Test Error

REGIME 2 (High Bias)

How to recognize?

- Training error is higher than ϵ .

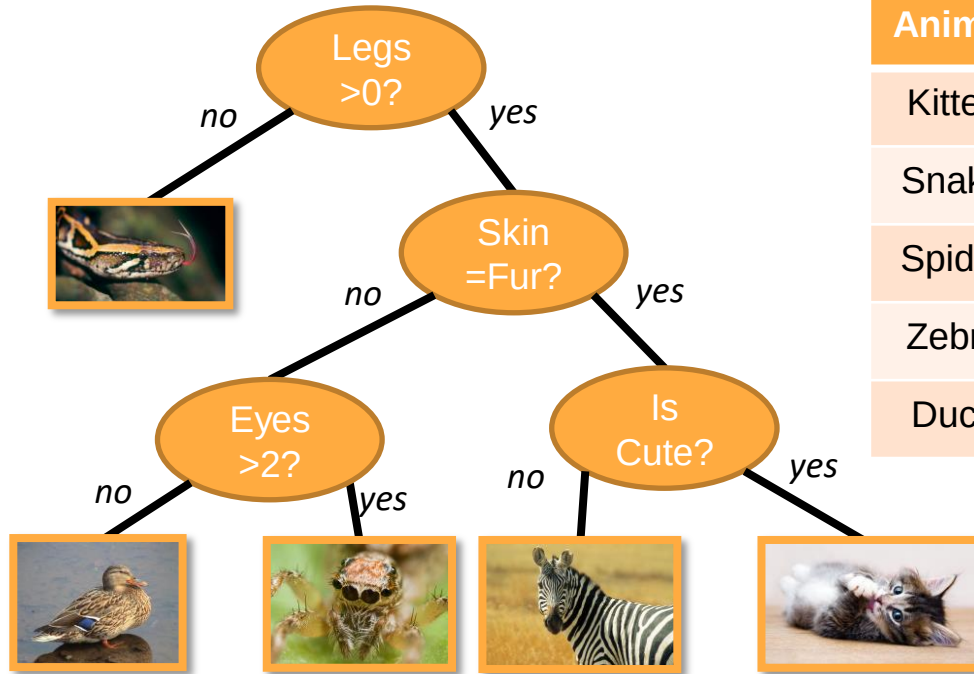
Suggestions:

- Use more complex model,
- Add features,
- Boosting.

Introduction

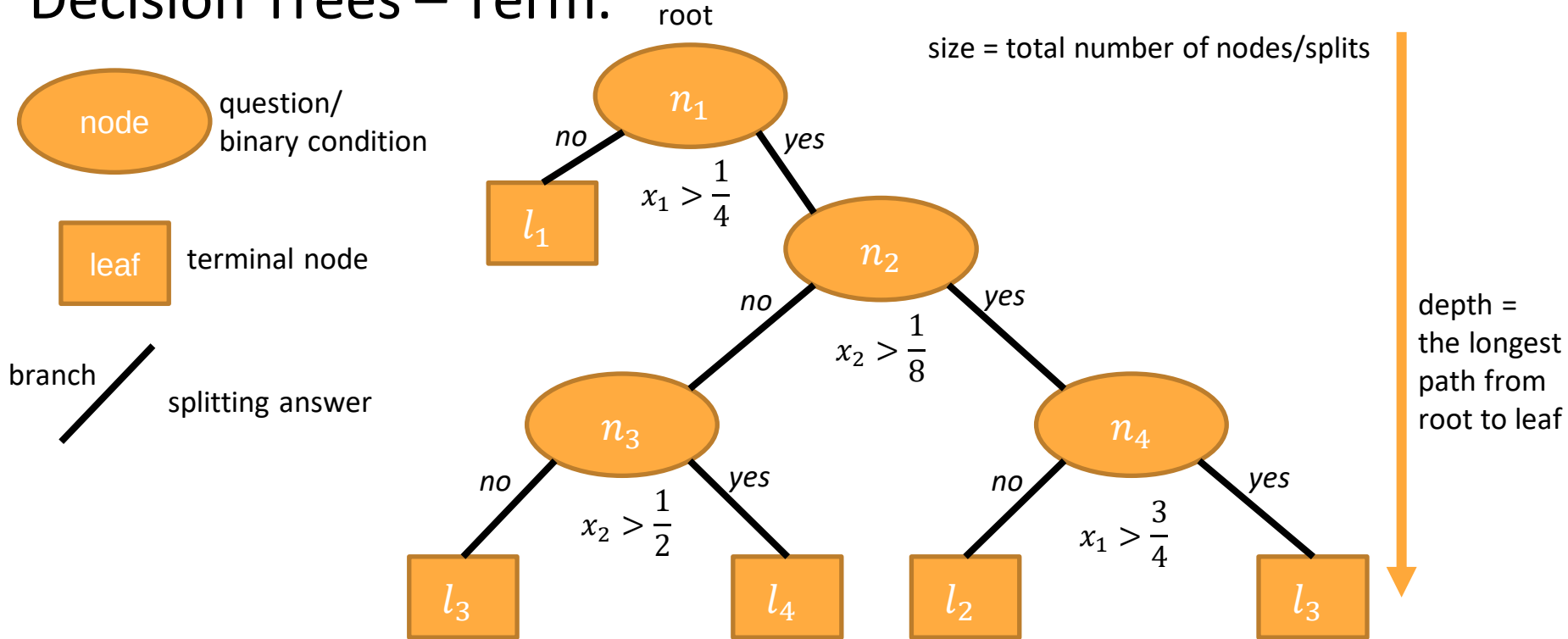
- Bias-Variance Tradeoff and Noise
- **Decision Trees**
 - basic principle
 - terminology
- CART (Classification And Regression Trees) Breiman et al. (1984)
 - splitting criterion (impurity measure)
 - stop-splitting rule
 - leaf assignment rule
- Bootstrap aggregated (bagged) trees
- Random Forest
- Adaptive Boosting (AdaBoost)
- Feature Importance with Decision Trees and Random Forest
- Another CART modifications – ID3 (Iterative Dichotomiser 3), C4.5, C5.0, CHAID (Chi-squared Automatic Interaction Detection), MARS (Multivariate Adaptive Regression Splines), etc.

Decision Trees – Basic principle



Animal	Skin	Legs	Eyes	Cute
Kitten	'Fur'	4	2	True
Snake	'Scales'	0	2	False
Spider	'Chitin'	8	8	False
Zebra	'Fur'	4	2	False
Duck	'Feather'	2	2	False

Decision Trees – Term.

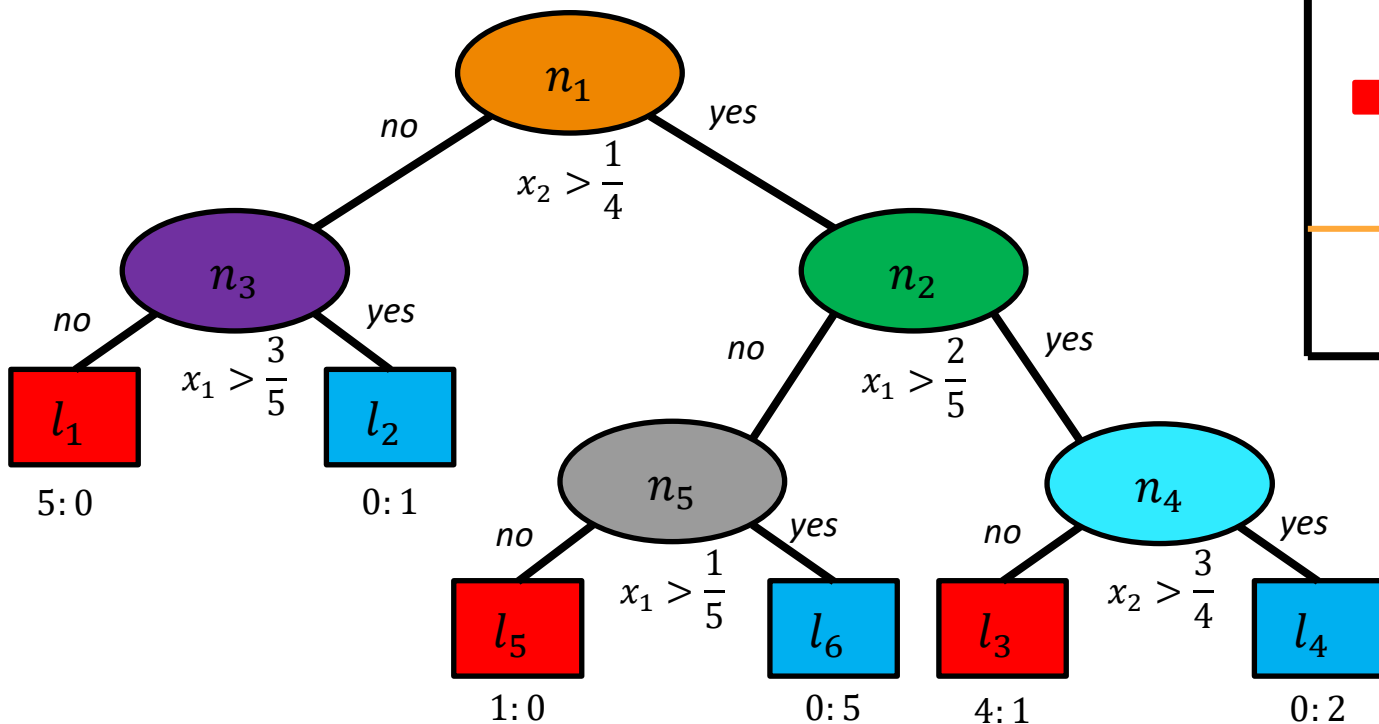


note: tree with one node only – “stump”

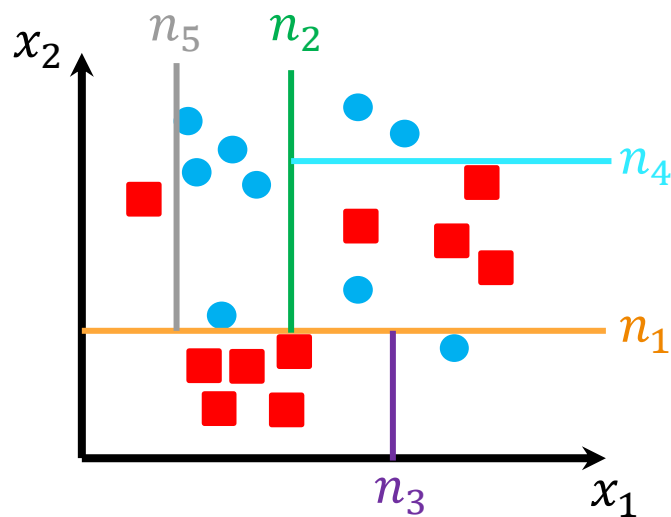
Introduction

- Bias-Variance Tradeoff and Noise
- Decision Trees
 - basic principle
 - terminology
- **CART (Classification And Regression Trees) Breiman et al. (1984)**
 - splitting criterion (impurity measure)
 - stop-splitting rule
 - leaf assignment rule
- Bootstrap aggregated (bagged) trees
- Random Forest
- Adaptive Boosting (AdaBoost)
- Feature Importance with Decision Trees and Random Forest
- Another CART modifications – ID3 (Iterative Dichotomiser 3), C4.5, C5.0, CHAID (Chi-squared Automatic Interaction Detection), MARS (Multivariate Adaptive Regression Splines), etc.

CART – Classification



Decision Trees



$$T = \{(x_1, y_1), \dots, (x_n, y_n)\}$$

$$x_i = \{x_1, \dots, x_D\}$$

$$x_{id} \in \mathbb{R} \quad y_i \in Y$$

Y – set of classes

assignment a specific class of each leaf – result by majority vote – y

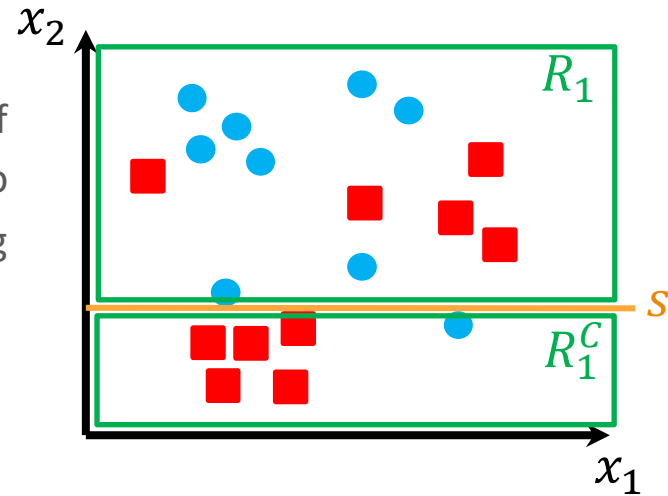
CART – Class. tree growing (train)

Task: minimize classification error E_k (number of misclassified points) in k_{th} node given by two complementary regions R_k and R_k^C split by splitting threshold s for d_{th} feature – splitting criterion:

$$E_k = \underset{\substack{d \in \{1, 2, \dots, D\} \\ s \in \mathbb{R}}}{\operatorname{argmin}} \left[E_{R_k(d, s)} + E_{R_k^C(d, s)} \right],$$

$$R_k(d, s) = \{\mathbf{x}_i : x_{id} > s\} \quad R_k^C(d, s) = \{\mathbf{x}_i : x_{id} \leq s\}$$

$$E_{R_k} = \sum_{i: \mathbf{x}_i \in R_k} I(y_i \neq y), \quad I(.) - \text{indicator function with binary result}$$



CART – Class. tree growing (train)

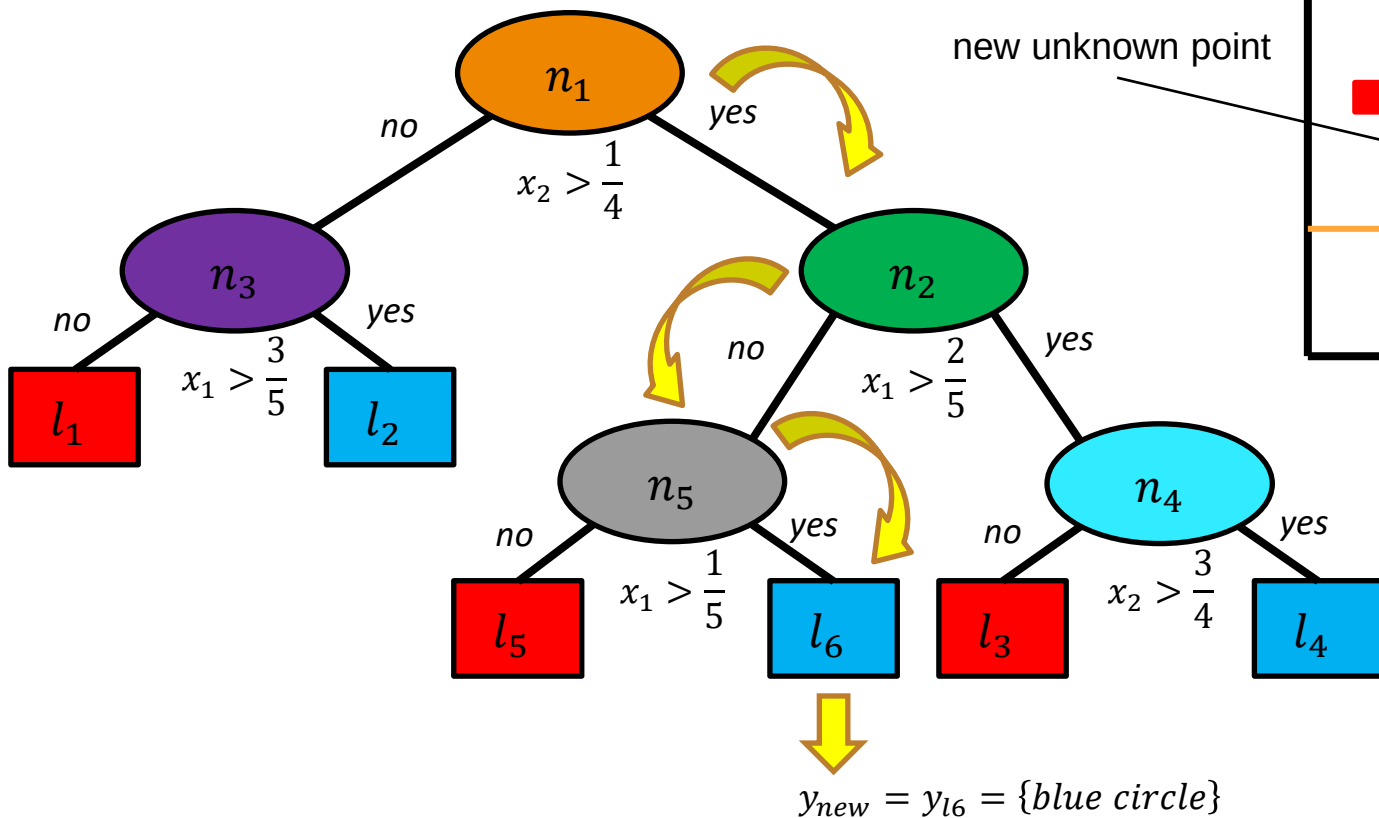
“Greedy” approach during tree growing

1. compute error E_k for each region R_k by selecting feature d and threshold s (by brute-force or histogram-based methods)
2. take the best region (with the lowest error) and split the data into two sub-regions in accordance with this feature and threshold
3. repeat for each new sub-region until stop-splitting criteria is reached

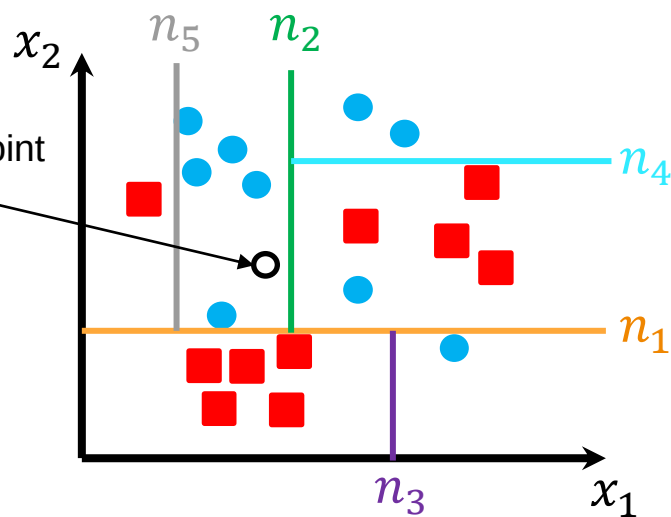
Stop-splitting rules:

- when one point in region
- split only if more than N points in region
- when region contains points of one class only
- maximum depth of tree reached
- maximum number of splits (tree size) reached

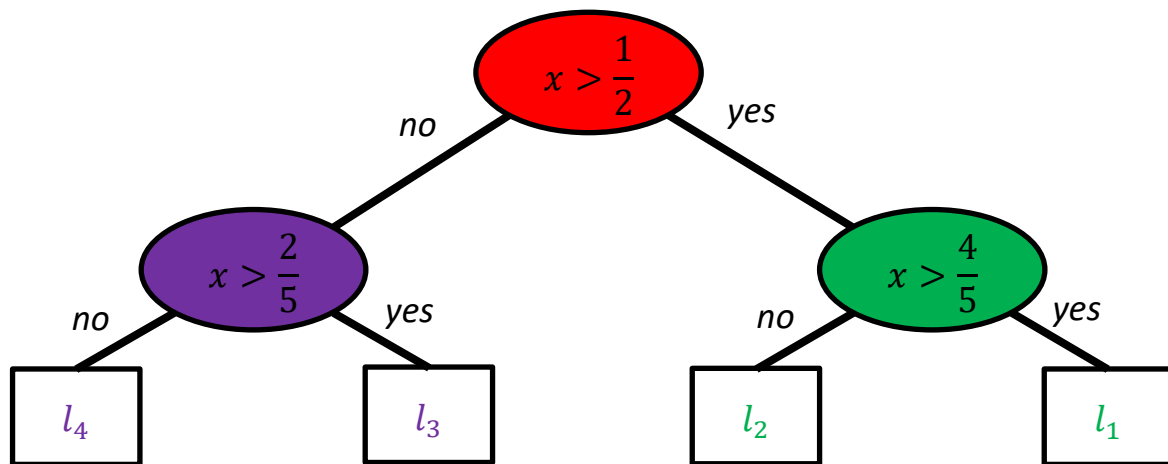
CART – Class. tree (classification)



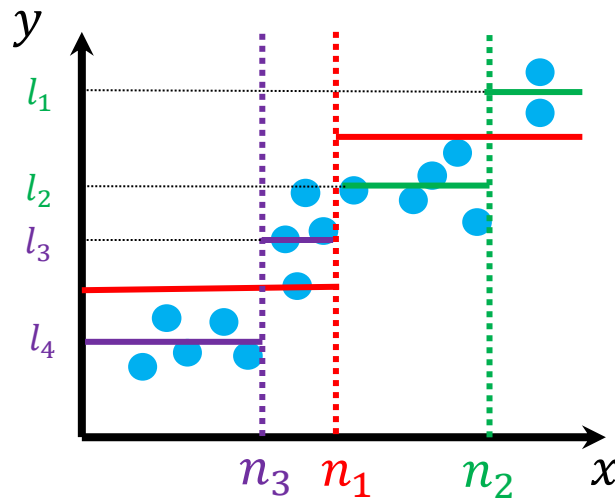
Decision Trees



CART – Regression



Decision Trees



Task: minimize error E_k in k_{th} node given by two complementary regions R_k and R_k^C splitted by splitting threshold s for d_{th} feature – splitting criterion: squared error

$$E_{R_k} = \sum_{i: x_i \in R_k} (y_i - y)^2$$

value of each leaf is determined by mean of points

CART – impurity measures

Impurity measures (splitting criteria):

- Classification

- Misclassification rate

$$E_{R_k} = \sum_{i: x_i \in R_k} I(y_i \neq y)$$

- Entropy

$$H_{R_k} = - \sum_{y \in Y} p_{R_k}(y) \cdot \log(p_{R_k}(y))$$

- "Gini impurity"

$$G_{R_k} = \sum_{y \in Y} p_{R_k}(y) \cdot (1 - p_{R_k}(y))$$

- Regression

- Squared error

$$E_{R_k} = \sum_{i: x_i \in R_k} (y_i - y)^2$$

CART – Conclusion

Pros:

- simple and intuitive,
- can handle numerical and categorical data,
- minimal requirements for data preprocessing (independent to scaling – no need of normalization),
- can handle missing variable (e.g. randomly),
- can handle correlated data.

Cons:

- instability – data dependent (can be solved by bootstrap aggregation, Random Forest or boosting),
- overfitting with complex tree (can be solved by bootstrap aggregation, Random Forest or boosting).

Introduction

- Bias-Variance Tradeoff and Noise
- Decision Trees
 - basic principle
 - terminology
- CART (Classification And Regression Trees) Breiman et al. (1984)
 - splitting criterion (impurity measure)
 - stop-splitting rule
 - leaf assignment rule
- Bootstrap aggregated (bagged) trees
- Random Forest
- Adaptive Boosting (AdaBoost)
- Feature Importance with Decision Trees and Random Forest
- Another CART modifications – ID3 (Iterative Dichotomiser 3), C4.5, C5.0, CHAID (Chi-squared Automatic Interaction Detection), MARS (Multivariate Adaptive Regression Splines), etc.

Boosting methods

- methods for combination of more weak classifiers into one stronger
- **Bootstrap Aggregation** (“Bagging”) – simple average
- **Adaptive Boosting** (“AdaBoost”) – weighted average
- and many modifications:
 - e.g. LPBoost, RUSBoost, RobustBoost, XBoost, etc.

Bootstrap Aggregation – “Bagging”

- standard regression tree:

$$T = \{(x_1, y_1), \dots, (x_n, y_n)\}$$

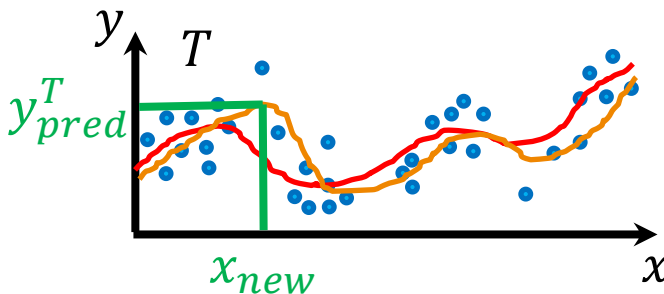
$$x_i \in \mathbb{R} \quad y_i \in \mathbb{R} \quad iid$$

$$E \left((y - y_{pred}^T)^2 \right) = \sigma^2(y)$$

- High variance problem:

$$E_{x,T} \left[\left(h_T(x) - \bar{h}(x) \right)^2 \right] \rightarrow high$$

$f(x) = y$ – ground truth
 $g(x) = y_{pred}^T$ – prediction
of tree trained on whole
dataset T



Bootstrap Aggregation/"Bagging"

- statistical technique to reduce the variance of a classification or regression procedure
- take $m \times n'$ random samples with replications from T with uniform distribution:

$$T_1 = \{(x_1^{(1)}, y_1^{(1)}), \dots, (x_{n'}^{(1)}, y_{n'}^{(1)})\}$$

$$\vdots$$

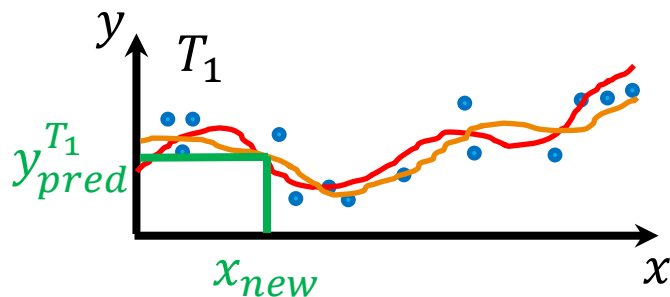
$$T_M = \{(x_1^{(M)}, y_1^{(M)}), \dots, (x_{n'}^{(M)}, y_{n'}^{(M)})\}$$

$$E\left((y - y_{pred}^{T_{bagg}})^2\right) = \frac{1}{M} \sigma^2(y), \quad \text{where } y_{pred}^{T_{bagg}} = \frac{1}{M} \sum_{m=1}^M y_{pred}^{T_m}$$

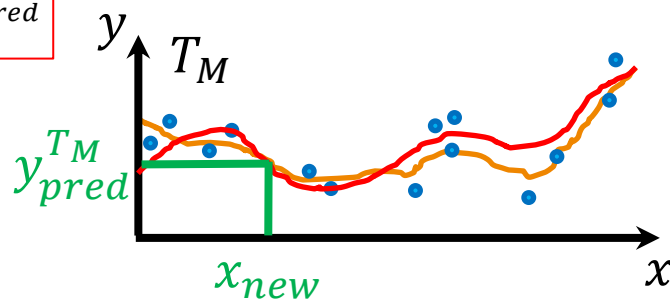
- usually – $n' = n$
- regression – average
- classification – majority vote

$f(x) = y$ – ground truth

$f(x) = y_{pred}^{T_m}$ – prediction of tree trained on dataset T_m



⋮

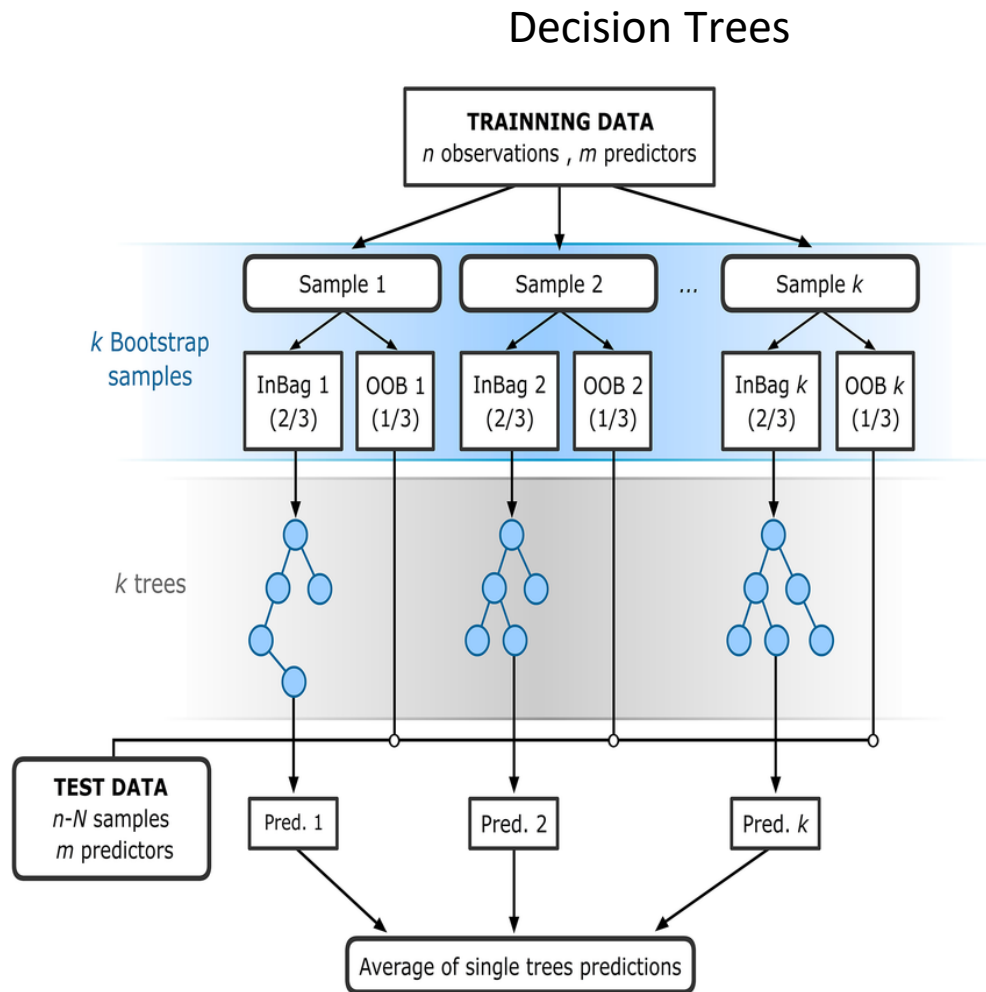


Introduction

- Bias-Variance Tradeoff and Noise
- Decision Trees
 - basic principle
 - terminology
- CART (Classification And Regression Trees) Breiman et al. (1984)
 - splitting criterion (impurity measure)
 - stop-splitting rule
 - leaf assignment rule
- Bootstrap aggregated (bagged) trees
- Random Forest
- Adaptive Boosting (AdaBoost)
- Feature Importance with Decision Trees and Random Forest
- Another CART modifications – ID3 (Iterative Dichotomiser 3), C4.5, C5.0, CHAID (Chi-squared Automatic Interaction Detection), MARS (Multivariate Adaptive Regression Splines), etc.

Random Forest

- bootstrap sample
- each node splitting – consider only random subset of features
- InBag sample – for training
- Out Of Bag (OOB) samples
 - testing data for validation
 - each tree has own OOB sample
 - not all trees are used for validation of each sample – only subset of trees with testing sample in their OOB set are used
 - advantage if we need all available dataset as training (small datasets)
 - generally OOB score is lower than validation score on independent validation dataset – OOB score and validation score are not comparable



Random Forest – Conclusion

Pros:

- same as CART,
- suitable even for small datasets (OOB samples),
- higher performance than single decision tree,
- performance is comparable with the best standard supervised learning algorithms,
- trustworthy test error estimate without need of the cross-validation (OOB samples)
- reliable feature importance estimate (later),
- can be easily parallelized,
- stable and prevents overfitting.

Cons:

- less interpretable and intuitive than single decision tree,
- large number of deep trees – high computational and memory demand in both, training and prediction phase,
- still can overfit on noisy and/or regression data.

Introduction

- Bias-Variance Tradeoff and Noise
- Decision Trees
 - basic principle
 - terminology
- CART (Classification And Regression Trees) Breiman et al. (1984)
 - splitting criterion (impurity measure)
 - stop-splitting rule
 - leaf assignment rule
- Bootstrap aggregated (bagged) trees
- Random Forest
- [Adaptive Boosting \(AdaBoost\)](#)
- Feature Importance with Decision Trees and Random Forest
- Another CART modifications – ID3 (Iterative Dichotomiser 3), C4.5, C5.0, CHAID (Chi-squared Automatic Interaction Detection), MARS (Multivariate Adaptive Regression Splines), etc.

Adaptive Boosting – “AdaBoost”

- linear combination of weak classifiers – weighted majority vote
- based on classification improving on weighted versions of data
- for binary decision with classes ± 1 :

$$H(\mathbf{x}) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(\mathbf{x}) \right)$$

- $\mathbf{x}$ – input data
- h – weak classifier
- t – index of weak classifier
- T – number of weak classifiers
- H – final strong (boosted) classifier
- sign – sign function with output $+1$ or -1
- α – weight of weak classifier – **need to find for each classifier → AdaBoostM1**

Adaptive Boosting – “AdaBoost”

$$d = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)\}$$

$$\mathbf{x}_n \in \mathbb{R} \quad y_i \in \{-1, +1\}$$

1. Initialize data weights as: $D_n^{(t=1)} = \frac{1}{N}$,
2. find the best weak classifier h_t from set of all classifiers $\mathcal{H}$ as: $h_t = \arg \min_{h_j \in \mathcal{H}} \epsilon_t$, where

$$\epsilon_t = \sum_{n: h_t(\mathbf{x}_n) \neq y_n} D_n^{(t)},$$

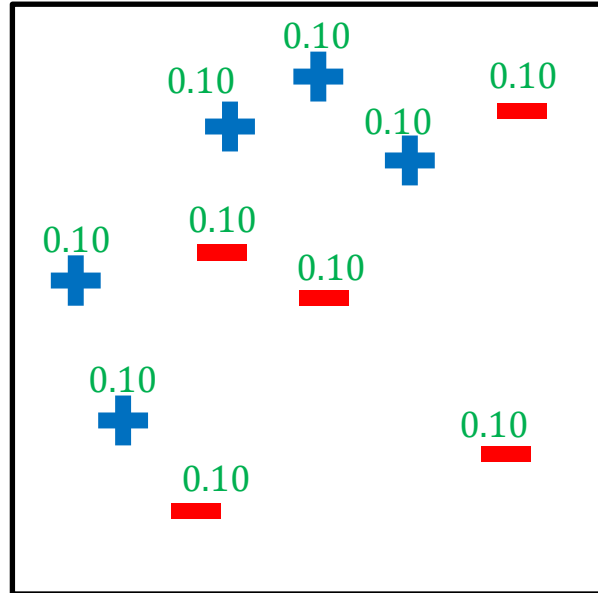
3. compute weight of weak classifier as: $\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right)$,
4. compute new data weights based on actual weak classifier and its weights as: $D_n^{(t+1)} = \frac{D_n^{(t)} e^{-\alpha_t y_n h_t(\mathbf{x}_n)}}{Z_t}$, where $Z_t = \sum_{n=1}^N D_n^{(t)} e^{-\alpha_t y_n h_t(\mathbf{x}_n)}$ is a normalization constant,

5. repeat steps 2. – 4. for all T weak classifiers.

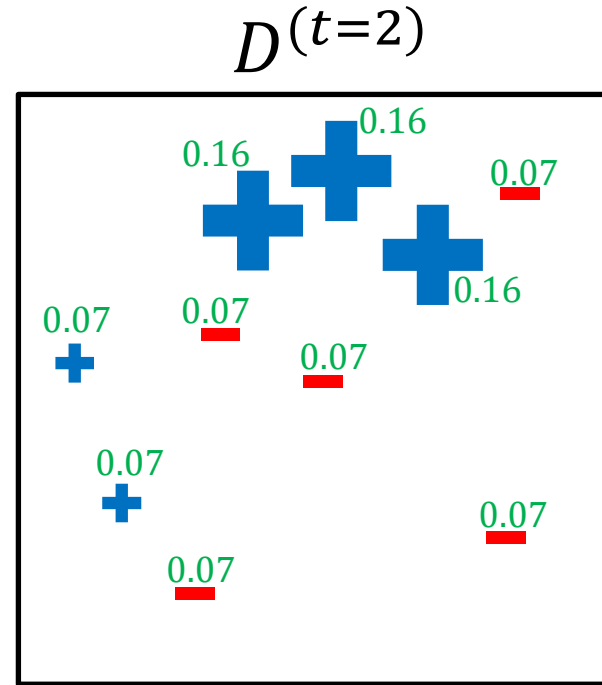
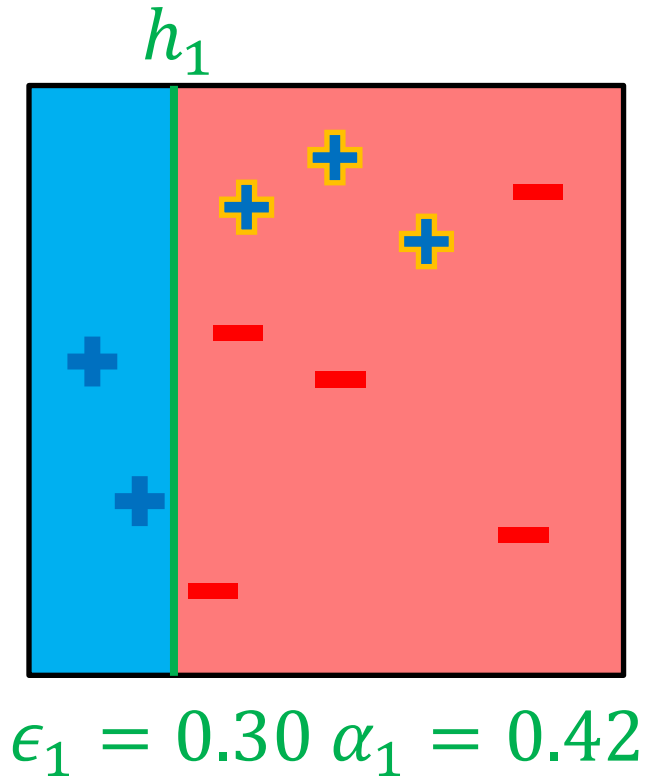
Adaptive Boosting – “AdaBoost” – Example

$$T = 3$$

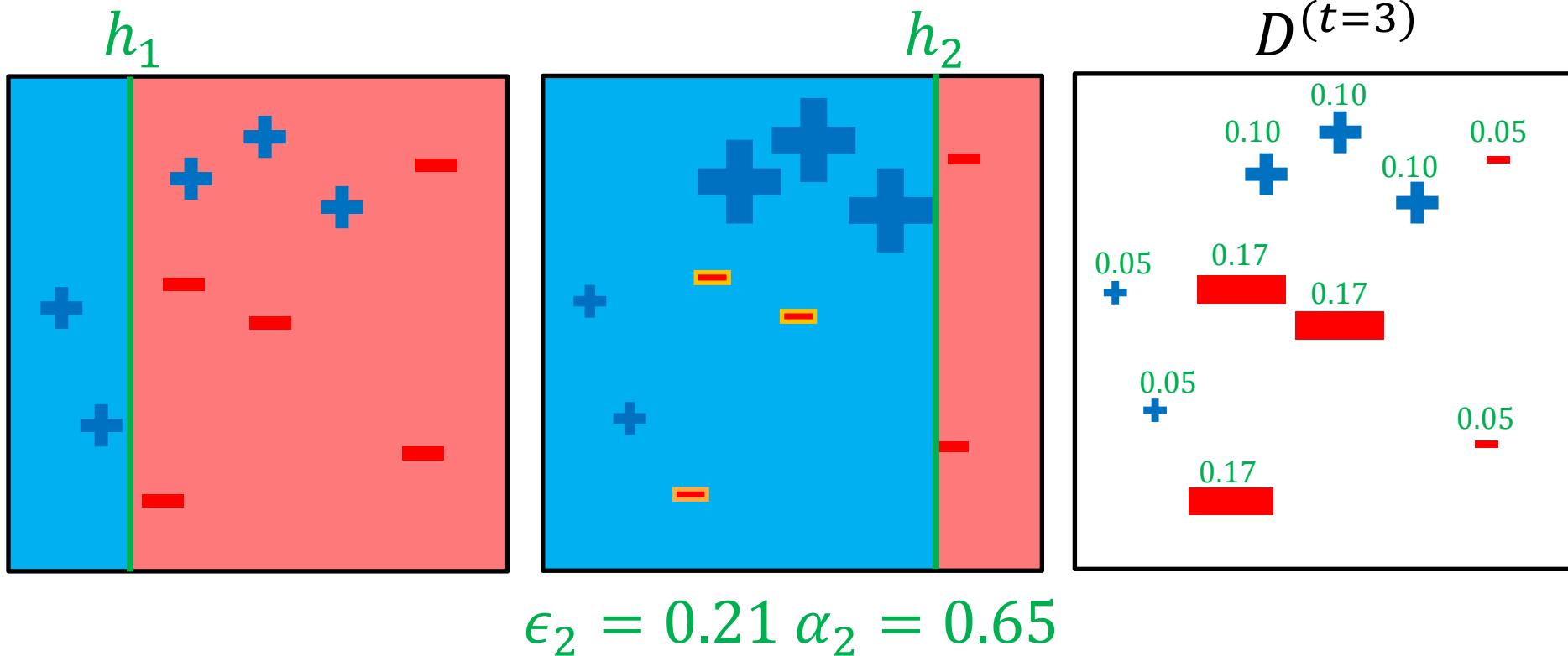
$$D_n^{(t=1)} = \frac{1}{N} = 0.10$$



Adaptive Boosting – “AdaBoost” – Example



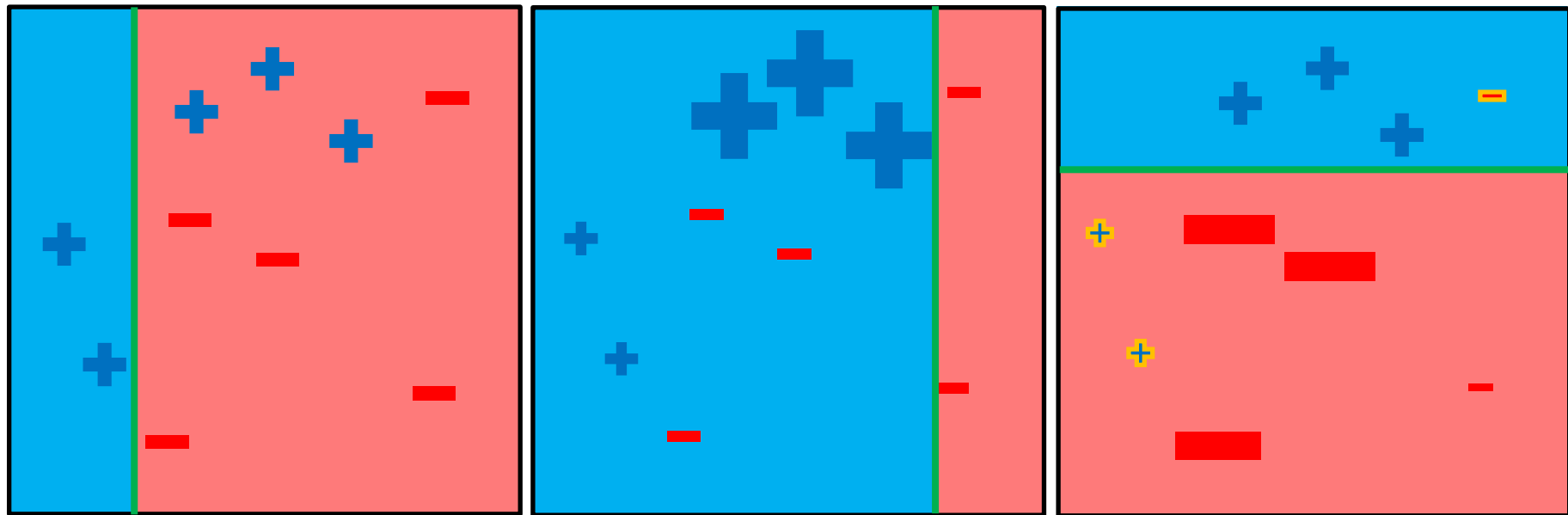
Adaptive Boosting – “AdaBoost” – Example



Adaptive Boosting – “AdaBoost” – Example

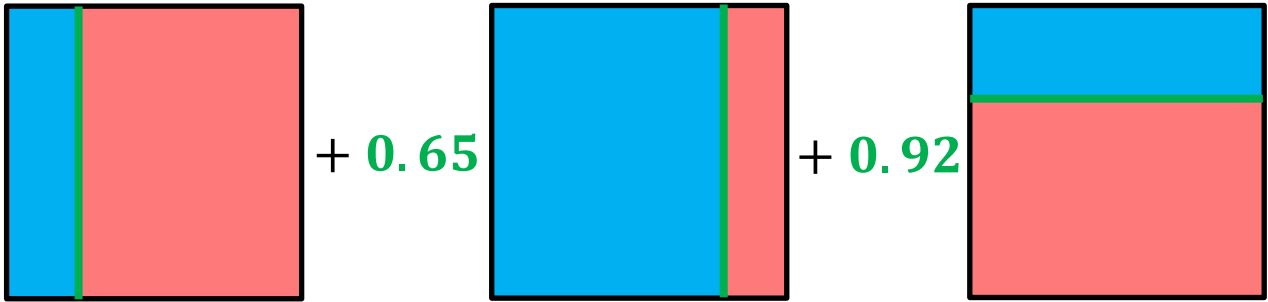
 h_1 h_2 h_3

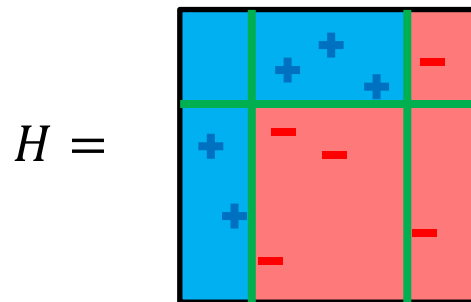
$$\epsilon_3 = 0.14 \quad \alpha__3 = 0.92$$



Adaptive Boosting – “AdaBoost” – Example

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right)$$

$$H = \text{sign} \left[0.42 \left(\text{left} \right) + 0.65 \left(\text{right} \right) + 0.92 \left(\text{top} \right) \right]$$




AdaBoost – Conclusion

Pros:

- fast and simple,
- can combine any ML algorithm,
- can combine expert decisions,
- no hyperparameter to tune (except number of iterations/weak classifiers and classifier type),
- can be extended for multiclass problem (AdaBoostM2).

Cons:

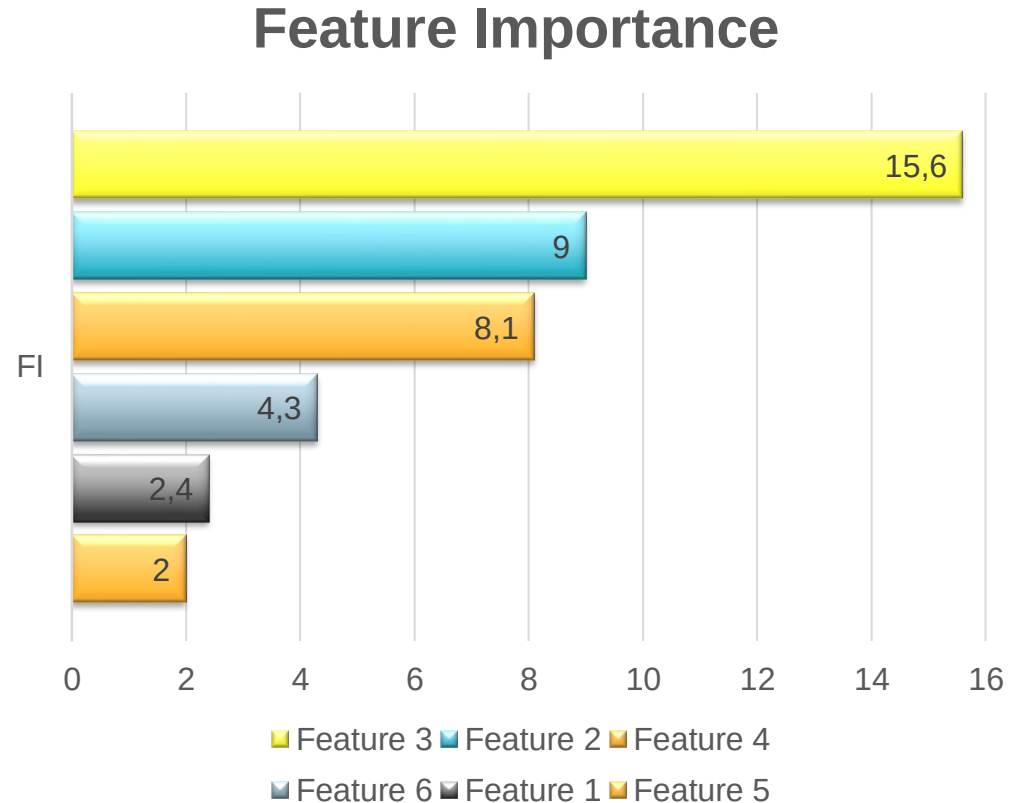
- tends to overfitting.

Introduction

- Bias-Variance Tradeoff and Noise
- Decision Trees
 - basic principle
 - terminology
- CART (Classification And Regression Trees) Breiman et al. (1984)
 - splitting criterion (impurity measure)
 - stop-splitting rule
 - leaf assignment rule
- Bootstrap aggregated (bagged) trees
- Random Forest
- Adaptive Boosting (AdaBoost)
- Feature Importance with Decision Trees and Random Forest
- Another CART modifications – ID3 (Iterative Dichotomiser 3), C4.5, C5.0, CHAID (Chi-squared Automatic Interaction Detection), MARS (Multivariate Adaptive Regression Splines), etc.

Decision trees – feature importance

- I. Impurity based
- II. Feature permutation
- III. Feature drop



Decision trees – feature importance

I. Impurity based

1) compute impurity at each node:

$$NI_k = w_k E_k - w_{R_k} E_{R_k} - w_{R_k^C} E_{R_k^C}, \text{ where } w_k = \frac{\|R_k\| + \|R_k^C\|}{N}$$

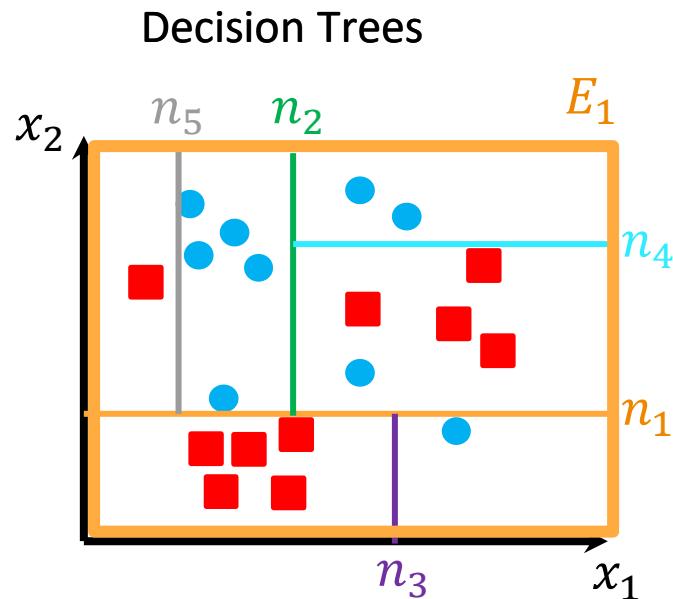
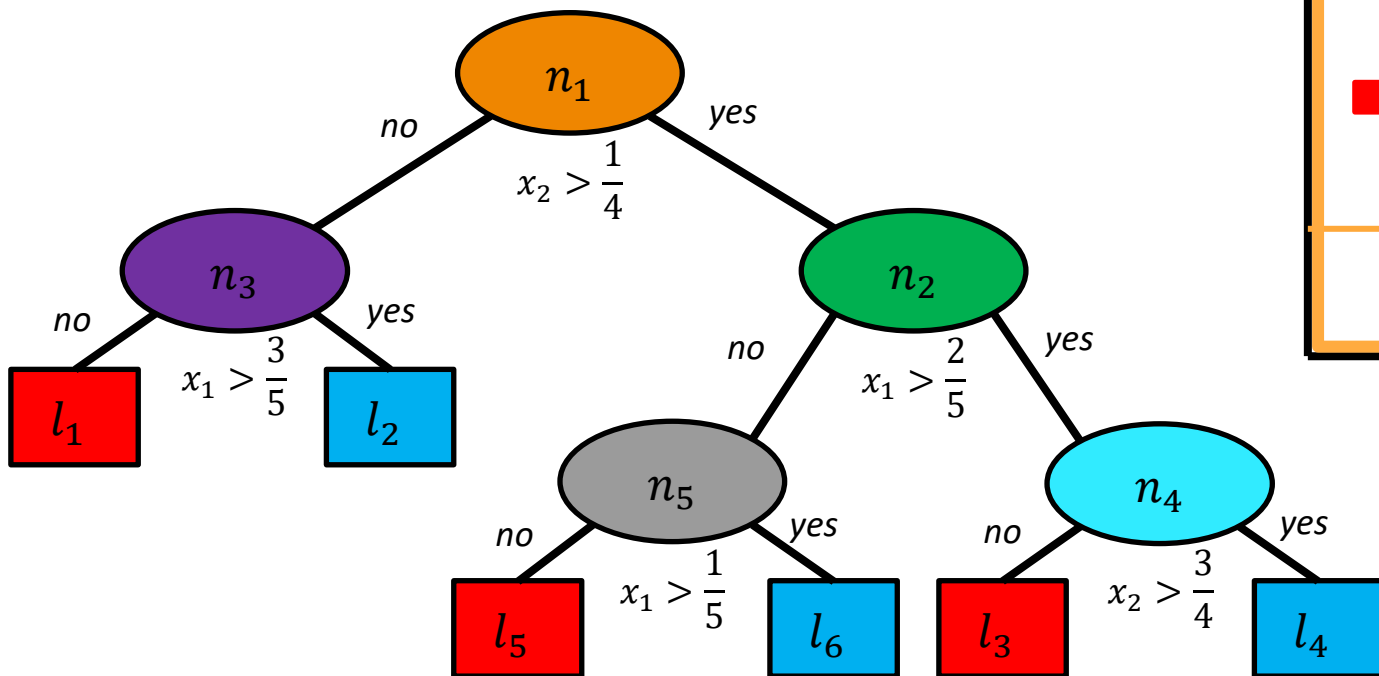
2) compute feature importance:

$$FI_d = \frac{\sum_{j: n_k \text{ splits on } d} NI_j}{\sum_{k=1}^K NI_k}$$

3) Random Forest FI:

$$RFFI_d = \frac{\sum_{j=1}^M FI_{dj}}{M}$$

Impurity based FI – Example

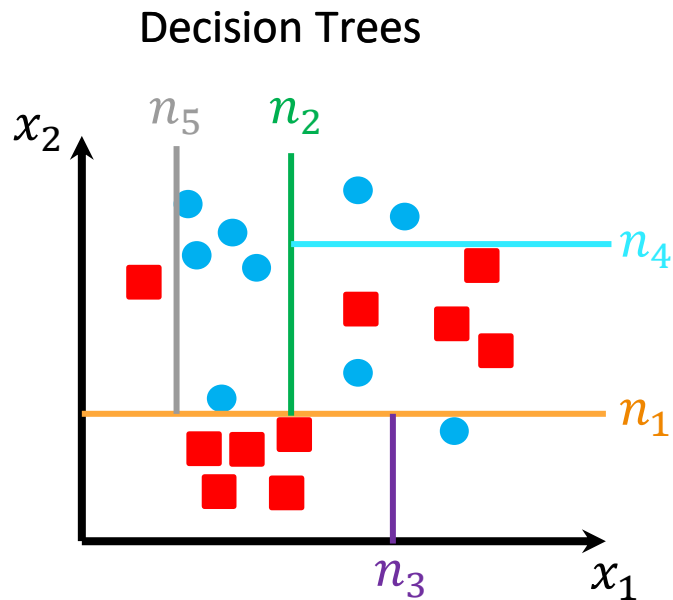
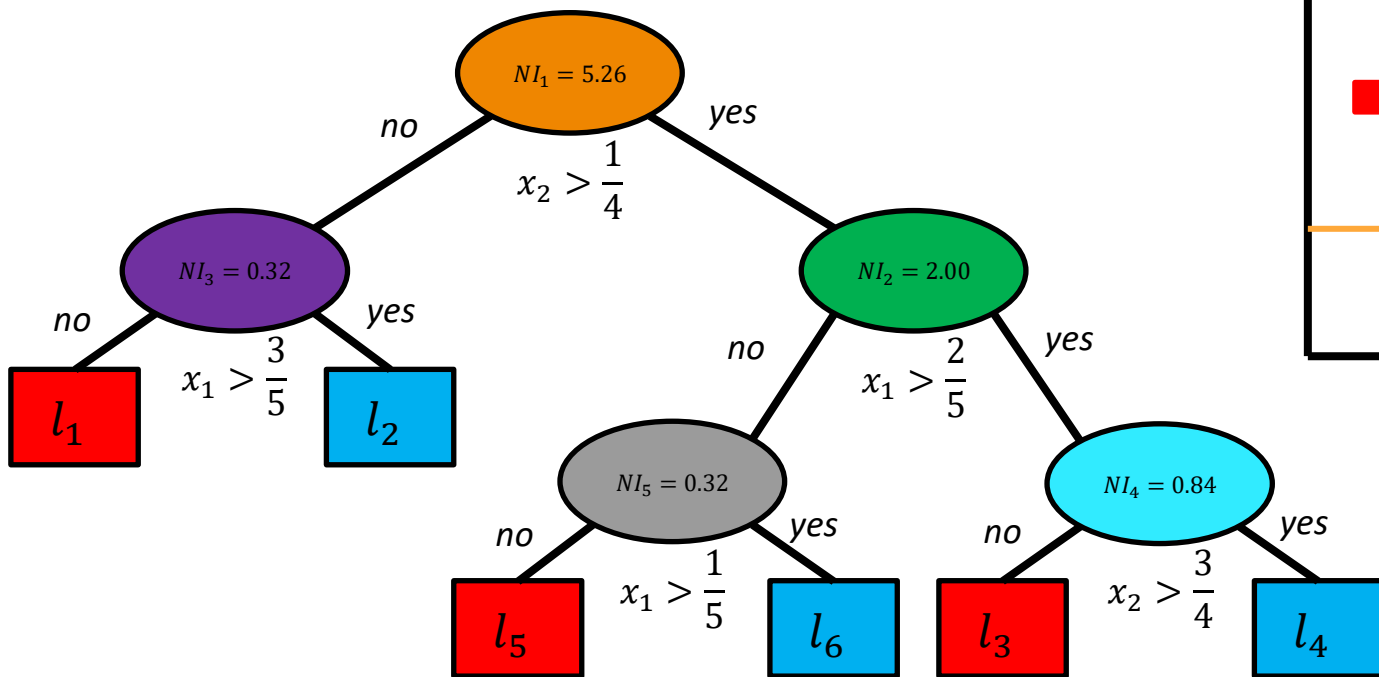


$$E_1 = \sum_{i: x_i \in R_1} I(y_i \neq y) = 9$$

$$w_1 = \frac{\|R_1\| + \|R_1^C\|}{N} = \frac{13 + 6}{19} = 1$$

$$NI_1 = w_1 E_1 - w_{R_1} E_{R_1} - w_{R_1^C} E_{R_1^C} = \frac{13 + 6}{19} (9) - \frac{13}{19} (5) - \frac{6}{19} (1) = 9 - 3.4211 - 0.3158 = \mathbf{5.2631}$$

Impurity based FI – Example



$$RFFI_d = \frac{\sum_{j=1}^M FI_{dj}}{M}$$

$$FI_d = \frac{\sum_{j:n_k \text{ splits on } d} NI_j}{\sum_{k=1}^K NI_k} \left\{ \begin{array}{l} FI_1 = \frac{NI_2 + NI_3 + NI_5}{NI_1 + \dots + NI_5} = \frac{2.00 + 0.32 + 0.32}{5.26 + 2.00 + 0.32 + 0.84 + 0.32} = 0.30 \\ FI_2 = \dots = 0.70 \end{array} \right.$$

Impurity based FI – Conclusion

Pros:

- fast calculation.

Cons:

- biased approach – inflate the importance of continuous features or high-cardinality categorical variable,
- highly dependent on samples and constructed tree – eliminated with random forest – mean over more trees

Decision trees – feature importance

II. Feature permutation

- 1) train a baseline model on training data
- 2) compute the quality (some appropriate metric – accuracy, MSE, ...) of the trained model on the training data (biased!), validation data or **Out Of Bag** set (for Random Forest)
- 3) shuffle values from one feature
- 4) predict output using dataset with shuffled values
- 5) compute the quality on shuffled dataset
- 6) compute FI as the difference between quality of the baseline model and result of model with shuffled dataset
- 7) repeat from 3) for each feature

Permutation based FI – Conclusion

Pros:

- applicable to any model,
- reasonably efficient,
- reliable technique,
- no need to retrain model at each modification of the dataset.

Cons:

- more computational complexity than the impurity-based method,
- overestimates correlated features.

Decision trees – feature importance

III. Feature drop

- 1) train a baseline model on training data
- 2) compute the quality (some appropriate metric – accuracy, MSE, ...) of the trained model on the training data (biased!), validation data or **Out Of Bag** set (for Random Forest)
- 3) drop out one feature from data
- 4) train new model without dropped feature
- 5) predict output using dataset with dropped feature
- 6) compute the quality of new model
- 7) compute FI as the difference between quality of the baseline model and new model
- 8) repeat from 3) for each feature

Drop out based FI – Conclusion

Pros:

- most accurate feature importance.

Cons:

- high computational complexity – re-training model for each feature drop.

Introduction

- Bias-Variance Tradeoff and Noise
- Decision Trees
 - basic principle
 - terminology
- CART (Classification And Regression Trees) Breiman et al. (1984)
 - splitting criterion (impurity measure)
 - stop-splitting rule
 - leaf assignment rule
- Bootstrap aggregated (bagged) trees
- Random Forest
- Adaptive Boosting (AdaBoost)
- Feature Importance with Decision Trees and Random Forest
- Another CART modifications – ID3 (Iterative Dichotomiser 3), C4.5, C5.0, CHAID (Chi-squared Automatic Interaction Detection), MARS (Multivariate Adaptive Regression Splines), etc.

Another CART modifications

- ID3 (Iterative Dichotomiser 3) Quinlan 1986
 - Information gain/entropy
 - Works with categorical variables only
 - 1. Compute total information,
 - 2. compute expected information for each feature,
 - 3. compute information gain for each feature,
 - 4. split with feature with maximal information gain,
 - 5. recursively repeat 2. – 4.
- C4.5/C5.0
 - Modification of ID3 – can also use numerical variables
- Pruning